Open in app ↗ Sign up Sign in

Medium Search

Booking.com Engin… · Follow publication

Unlocking the Power of Customization: How Our Enrichment System Transforms Recommendation Data Enrichments

7 min read · Jun 18, 2025

Juan Pablo Lorenzo Follow

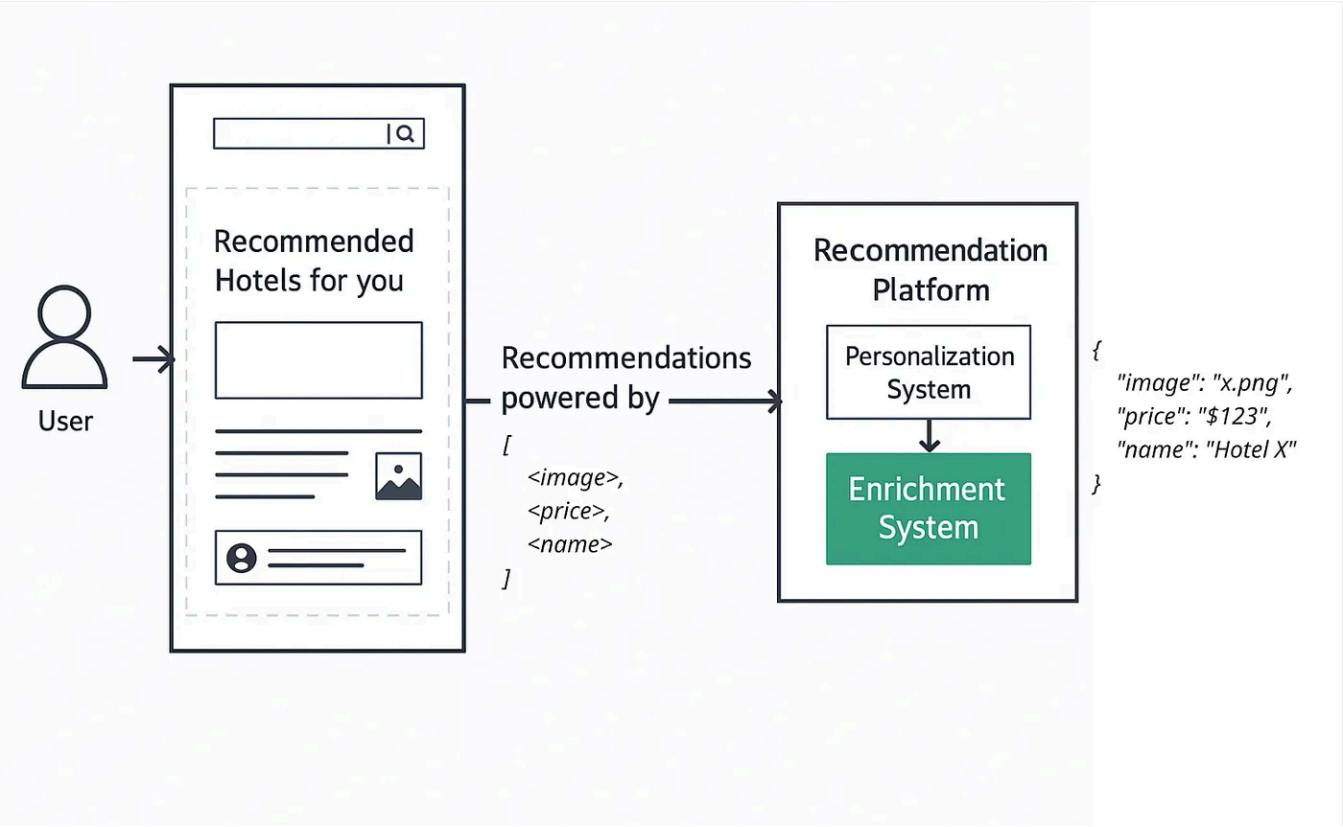
Listen Share

How are accurate property prices on Booking.com connected to machine learning that recommends appealing property photos? What about the number of users who have wishlisted a property? And how can developers assess if their recommendation models effectively boost traveler clicks? None of these pieces of information **are recommendations on their own**, but they're **crucial when providing our travelers good recommendations.** For years, our Recommendation Platform has handled this process, but we needed a better way.

Recommendation Platform is a service created to empower teams with adaptive, scalable, and personalized recommendations. These recommendations are integrated into every step of the customer journey, covering everything from attractions and flights to travel destinations and accommodation. We allow teams to use machine learning models combining multiple providers to achieve traveler recommendations. The platform is developed to be self-served and new use cases could be implemented by those teams. Check out our Self-Serve Platform for Scalable ML Recommendations article for a deeper look into how the platform works.

Recommendations are valuable by themselves, but in most cases, information about the specific recommendation is just as important. Previously, adding such

information was complex, lacked isolation, and had limited reusability. The new **Enrichment System** aims to overcome these challenges.



The Enrichment System is a **unified, extensible and scalable approach to support enrichments** in the Recommendation Platform. When using data to develop new capabilities, being fast and efficient is key. But our previous enrichment process often stood as a barrier rather than a facilitator. Here's why:

- **Complexity of implementation:** imagine a developer eager to enhance a recommendation with new data insights. Under the old system, implementing a new enrichment required **diving into a labyrinth of code modifications**. Each addition needed a lot of changes with the risk of introducing errors and inconsistencies between different customer journeys on Booking.com. This complexity undermined the platform's self-service design, slowed innovation, and required significant support from the Recommendation Platform team.

- **Tight coupling:** the previous architecture **intertwined enrichments with other components related to recommendations**, creating a tightly coupled environment. This dependency meant that a change in one part of the system could affect predictions made by the system, leading to unintended consequences and making troubleshooting a nightmare.

- **Limited reusability:** once an enrichment was developed, its **utility was often confined to a specific recommendation type** (e.g. Booking.com properties, trip destinations). The system's design didn't support easy reuse across different contexts. That's why similar enrichments had to be recreated from scratch multiple times, duplicating efforts and implementations.

The Enrichment System was born out of this necessity. It's designed to empower both developers and users by simplifying processes, reducing coupling, and promoting reusability.

The new approach: field masks and GraphQL

The cornerstone of our new Enrichment System is the adoption of **field masks** and the leverage of our already existing **GraphQL API**.

In APIs like ours, a field mask specifies which fields should be returned in a request, optimizing performance by reducing unnecessary data transfer.

In a GraphQL query:

```
query {
  user(id: "123") {
    name
    email
  }
}
```

Only the name and email fields of the user are returned.

Similarly, in a REST API you could use field masks like: *GET /v1/users/123?fields=name,email*. This request retrieves only the specified fields, reducing payload size and improving efficiency.

Ease of implementation

With the new system, users can now **request enrichments directly via field masks.** This eliminates the need to delve into intricate configurations and modify code.

Consider the example of requesting recommended images for a property. When the Enrichment System processes this request, it **checks for the presence of** *ENRICHED_IMAGES* **field mask.** If found, it **executes the corresponding enrichment in the Enrich stage** (find more information about the specific stages in the Recommendation Platform post). The enriched data is then attached to the response.

Get Juan Pablo Lorenzo's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

This **on-demand nature** of enrichment execution — driven by user requests through field masks — is the **preferred approach for new enrichments**. It **detaches the enrichment execution from the prediction stage**, making the process more intuitive and less prone to errors. Implementing a new enrichment becomes as straightforward as defining the field mask and its associated logic, rather than navigating a labyrinth of code modifications. On the other hand, this separation also reduces the prediction system's load, allowing for faster and more scalable inference.

Isolation and reusability

The new system inherently **reduces coupling** between enrichments and other parts of the platform. Because enrichments are triggered by specific field mask requests, changes to one enrichment are **less likely to have unintended consequences** on other functionalities. This modularity makes the system more stable and easier to maintain.

The design also promotes **reusability** of enrichments **through the platform**. Once an enrichment is implemented and associated with a field mask, it can be easily requested in various use cases simply by including that field mask in the data request. This eliminates the need to recreate similar enrichments for different applications. The focus on field masks allows teams to leverage existing enrichments in a standardized way, promoting efficiency and reducing duplicated efforts.

In essence, field masks act as clear and concise contracts for requesting specific enriched data, leading to a more streamlined, robust, and efficient enrichment process.

Feature	Former enrichments	Enrichment System
Implementation	Complex, required prediction code modifications	Simple, requested via field masks
Coupling	Tightly coupled with prediction stage	Reduced coupling, triggered by specific field mask requests
Reusability	Limited, often specific to one recommendation type (e.g. Properties)	Promotes reusability across the platform through field masks
Execution trigger	Implicit as part of the pipeline	On-demand, driven by user requests through field masks, detached from the prediction stage

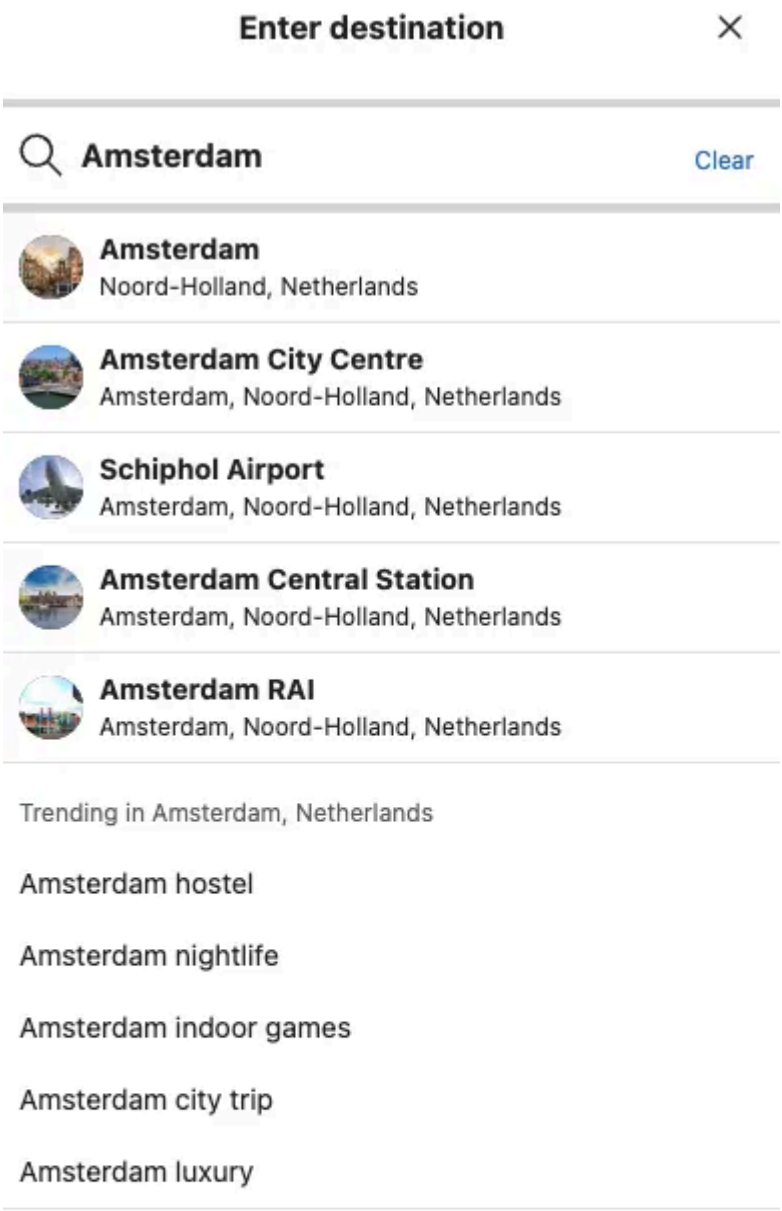
Real-world scenarios

To illustrate the power and flexibility of the new Enrichment System, let's look at a few concrete examples of enrichments:

- **Minimum Accurate Pricing enrichment:** this enrichment is designed to provide **precise pricing information** for recommendations. The system achieves this by **fetching data from a personalized ranker city search system.** Through the Enrichment System, users can specifically request this Minimum Accurate Pricing enrichment, ensuring they receive the most up-to-date and relevant price details without needing to retrieve unnecessary data.

- **Number of Wishlists enrichment:** imagine a user wanting to understand the popularity of a recommended property. The Number of Wishlists enrichment directly addresses this by **retrieving the number of times a property has been added to user wishlists**. The ease with which this specific piece of data can be requested through the new system **showcases the ability to tailor data retrieval to specific needs**, highlighting the flexibility introduced by field masks.

- **Recommended Images enrichment:** visual appeal is crucial in recommendations. The Recommended Images enrichment allows users to **request the best images relevant to their specific use case**. By requesting this enrichment, the system can deliver **optimized visuals**, thereby enhancing the user experience and the effectiveness of the recommendations. This process involves the use of a field mask like RECOMMENDED_IMAGES, which triggers the enrichment process.



- **Tracking Actions enrichment:** understanding user interaction with recommendations is vital for continuous improvement. It's important to track interactions like when recommendations are clicked, hovered over, or simply displayed to the traveler. The Tracking Actions enrichment focuses on **enabling frontend teams to integrate standardized tracking actions into their systems.** By providing a consistent way to request and receive tracking information, the

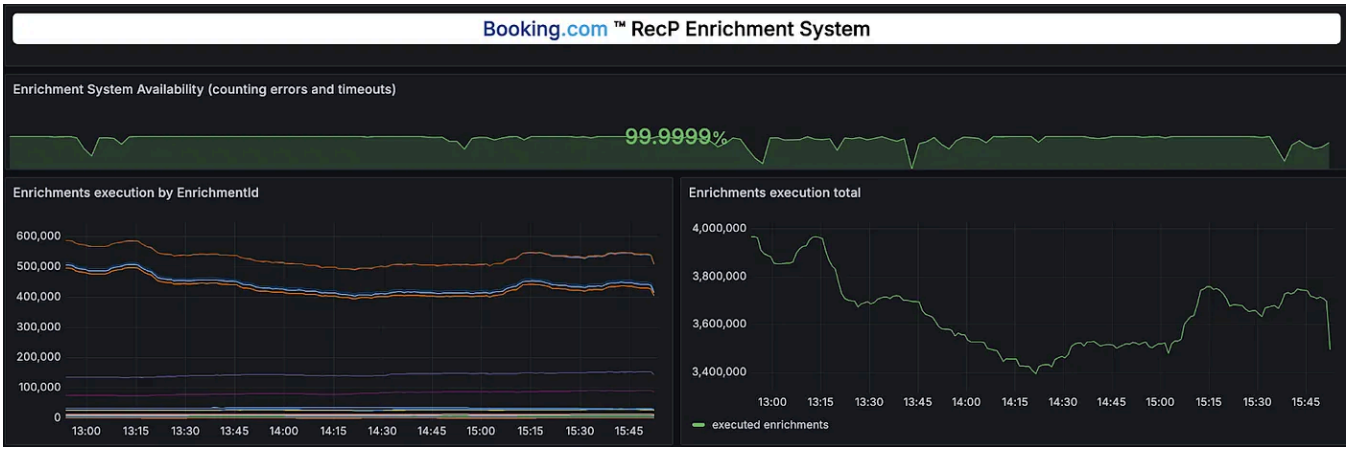
Enrichment System **simplifies data collection and reduces the need for complex tracking logic on the frontend.** This standardization allows for more efficient analysis of how users engage with recommendations, ultimately informing future optimizations.

Benefits of the new system

The adoption of the new Enrichment System brought significant advantages, particularly in terms of standardization, flexibility, performance, and alignment with industry best practices.

Standardization and flexibility: the new system leverages **field masks**, introducing a **standardized way to request data**, eliminating the complexities of previous configurations and reducing the potential for errors. By enabling users to define the enrichments they need through field masks, the system offers **greater flexibility and customization** in data retrieval. This on-demand nature of enrichments, driven by user requests, detaches the enrichment execution from the prediction stage.

Improved performance and reliability: the Enrichment System has already demonstrated its capability by **executing up to 100,000 enrichments per second with 99.99% availability**. This is supported by a robust **observability dashboard** that allows developers to monitor everything from the number of executions done by the system to each enrichment latency and failure rate. This highlights a significant improvement in both the performance and reliability of data enrichment processes.



Industry best practices: the use of **field masks for selective data retrieval** aligns with industry trends that emphasize optimized resource usage and improved performance. This method ensures that only the necessary data is fetched and processed, contributing to the overall efficiency of the system.

Conclusion

In conclusion, the new Enrichment System marks a significant leap forward in how we handle data enrichment within the Recommendation Platform. By adopting **field masks**, we have achieved **ease of implementation**, allowing developers to request specific data without navigating complex configurations. The system's design promotes **isolation and reusability** of enrichments, breaking free from the tightly coupled architecture of the past and preventing duplicated efforts. The capability to handle **up to 100,000 enrichments per second with 99.99% availability** also signifies a substantial improvement in **performance**.

We encourage you to explore how an approach similar to the Enrichment System can be leveraged for your specific needs. Its flexibility and efficiency have the potential to **transform your data processing workflows and unlock new levels of customization** in your recommendations. If you're building recommendation systems, consider how modular enrichments can accelerate your development. Feel free to reach out or share your thoughts below.

AI | Machine Learning | Java | Recommendation System | Software Development

Follow

Published in Booking.com Engineering

1.7K followers · Last published Feb 19, 2026

Software engineering at Booking.com

Follow

Written by Juan Pablo Lorenzo

5 followers · 16 following

Engineering @ Booking.com.